Project Atlas: Agent Orchestration in Tempest

Overview

Tempest has an incredible opportunity to pioneer the mass market adoption of AI in software operations and delivery. This is a space that is traditionally dominated by automation, but many existing automation strategies are starting to lag behind the sheer volume, complexity, and risk created by the use of AI by human engineers, and soon, the use of autonomous agents.

Tempest will become the platform for human/AI collaboration in software delivery. We have significant tailwinds in this area, given that our core platform has the ability to:

- Centrally catalog all infrastructure and services
- Authenticate securely and interact with infrastructure and services
- Orchestrate complex multi-step dependent workflows

To take full advantage of this opportunity, we must evolve our currently platform, which is largely focused on self-service deployments, in one that extends that to support:

- Listening to external events, processing them, and using them as stimulus into our Tempest orchestration
- The registration of expansive networks of arbitrary specialized AI agents
- The ability to orchestrate those agents to work together collaboratively (or exclusively) either autonomously
- Organizational and enterprise grade management of:
    - Agent access management
    - Agent memory

    - Lifecycle and upgrades
    - Agent Scratchpads
- Quick access to data summary via text and visualization

Wishlist

- Main workspace coordinator agent
    - In most cases, this should be the only workspace to have access to global memory and context
    - Manages a hierarchy of network-enabled functional sub-agents
    - Configurable agent discovery
    - Federation across Tempest and self-hosted agents
    - Tunable x-agent consistency for context & memory
- Support for signals
    - Drop in and event bindings for stimulus for external systems
    - Configurable with filtering and aggregation functions
- Self-hostable—the whole system should be built ground up to be self-hostable
    - Persistence adapters
    - Async messaging adapters
- Clear input/output data contracts for the system—eg no part of this system should have access to our core databases—data and configuration must be fed into it
- Workspace context
    - Always on, always fresh
    - Bound by configurable retention policy (pursuant to plan)
- Memory

    - Granular controls for agent memory. Temporary vs Permanent vs Time-bound
        - Levels:
            - Per workspace
            - Per agent per workspace
            - Per time-bound retention policy
- Audit
    - Agent & tool call audit
    - Memory/context audit
    - Query audit
    - User query sentiment analysis (Enterprise feature)
- Deployments
    - Existing deployment machine capabilities added to workspace
- Single workspaces
- Publish as workspace template
- Owners
- Governance/tags/relationships
- Tempest platform improvements:
    - Self-service workspace templates
    - Agent catalog
- Use outside of Tempest
    - MCP gateway
        - Tempest can act as a gateway for all the MCP servers within a workspace—reducing access risk and hydrating with more context and reducing MCP sprawl fatigue
            - Single MCP endpoint + def per workspace

            - Auditable workspace MCP usage & calls
            - Auditable MCP users
        - Agent Check API
            - Workspace agents can be bound to endpoints, for use as intelligent checks in GitHub Actions, etc..,
                - Results can be hydrated in workspace
- Errors and debugging
    - Workspace logs
    - Agent logs
    - Deployment logs
    - Signal logs
    - Access logs
- Native workspace metrics
    - Deployments
    - Signals
    - Agent invocations
        - Agent success rate
        - Human approval rate
- Gates
    - Workspace policies for deployments and agents
    - Answer verification gates
    - Human-in-the-loop ack and approvals
    - Cross agent gates

Project Atlas

Goals

Transform Tempest into a powerful, AI-native software delivery platform where engineers can create workspaces to collaborate seamlessly with specialized, autonomous agents.

Technical Goals

Build a software system that can orchestrate the lifecycle, state, memory, and communication between multiple AI agents, humans, and pre-defined deterministic workflows.

First Principles

First principles are fundamental, axiomatic truths that are true beyond this work.

1. The AI landscape will be rapidly growing in the next 12 months across models, tooling, and practical applications.
2. Broad traction for AI as coding agents and amongst software developers who regularly write code has been demonstrated (understatement of the century)
3. Traction amongst software delivery & cloud operations has been quieter, e.g, few well publicized examples and use cases.
4. Writing code is inherently more flexible and subjective than shipping software for most organizations
    a. While there may be many ways to implement a feature, deployment environments are often fixed specifications, and exist statically beyond any feature or code
5. Humans must sleep, eat, and get sick. Artificial agents can produce and refine indefinitely barring cost.
6. Today's LLMs are inherently non-deterministic, but can potentially achieve functional determinism via model, weights, prompt, controlled sampling and hardware.
7. The output of an LLM is a sequence of tokens

8. Today's LLMs have emergent capabilities—and the pace of new capabilities emerging has been breakneck since GPT-3

System Principles

Core values and assumptions via Tempest engineering and industry best practices.

1. LLMs perform better given more training, memory, and context. Training is an offline/compile-time concern and memory & context are a runtime concern.
2. Agents should largely be stateless. Memory, context, and scratchpad should be either 1) retrieved from storage or 2) ephemeral.
3. A workspace is an actor system, wherein each actor represents an agent
4. Each workspace should have a top-level agent: `WorkspaceSupervisor`
    a. The `WorkspaceSupervisor` is responsible for workspace configuration, managing agent actor lifecycle, and processing signal interrupts and lifecycle
5. `Signals` can be added to workspaces. Signals are external events with an optional condition, filter, or aggregation function applied.
6. Each `Signal` should be processed independently. Signals may be configured for idempotency:
    a. Signals of the same type may be configured for:
        i. Queueing behavior (process one-at-a-time)
        ii. Parallel processing
        iii. Replace/cancel
    b. Signals may interrupt other signals:
        i. Chain cancellations
        ii. Chain triggers
7. Each `Signal` spawns a `WorkspaceSession`. These sessions create unique context per signal, and spawn isolated instances of workspace agent actors. Think of these sessions as a "New Chat" in ChatGPT or Cursor

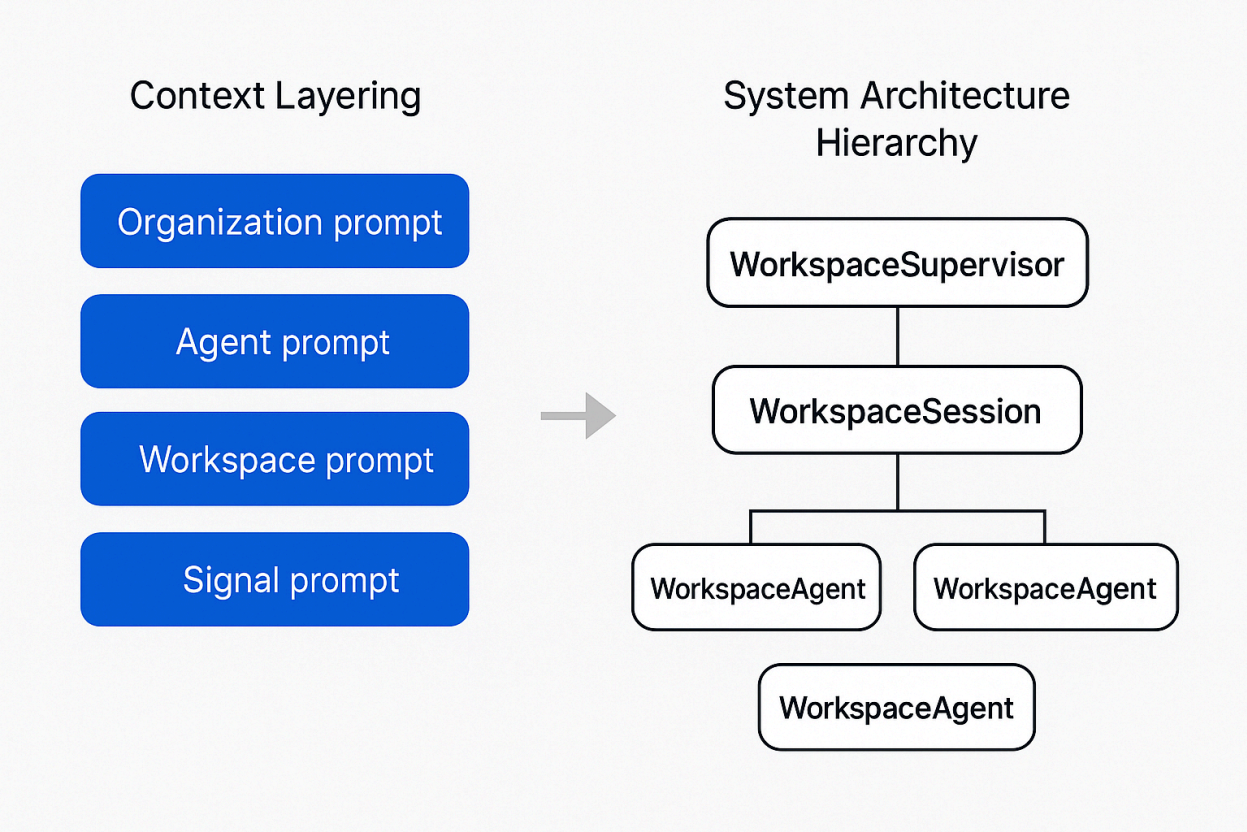
8. `WorkspaceSession` reads `WorkspaceConfiguration` and spawns the `WorkspaceAgents` for the `Signal` being processed for a given session.
9. Depending on the workspace's signal configuration, multiple sessions may be active. This should be configurable, as to enable commercial plan segmentation and throughput control
10. `Signals` should be manually triggerable by the user for testing purposes
11. `Signals` should be configurable via workspace specific prompt or client configuration
12. A `WorkspaceAgent` is an instance of a `TempestAgent`
13. `TempestAgents` are stateless actors. Context and memory, if configured to be persistent, will be loaded from storage priority to agent invocation
14. `TempestAgent` may be compiled with a prompt
15. `TempestAgent` may have a agent level prompt set via organization configuration
16. `TempestAgent` may have a workspace prompt set via `WorkspaceAgent` configuration
17. `TempestAgent` may have a workspace:signal specific prompt set via `Signal` configuration
18. All components of a workspace should be observed and trace
19. All mutations and invocations for a workspace should be attributed and audit logged
20. All cost-generating factors of a workspace should be tracked
21. In addition to `Signals`, workspaces can configure (as a sibling to signals and agents in the information architecture of setting up a workspace):
    a. `Insights` (Select a data source and matching visualization)
    b. `Workflows` (previously recipes)—deterministic multi-step workflows for anything from user to infrastructure provisioning
    c. `People`
        i. `Owners`
        ii. `Watchers`

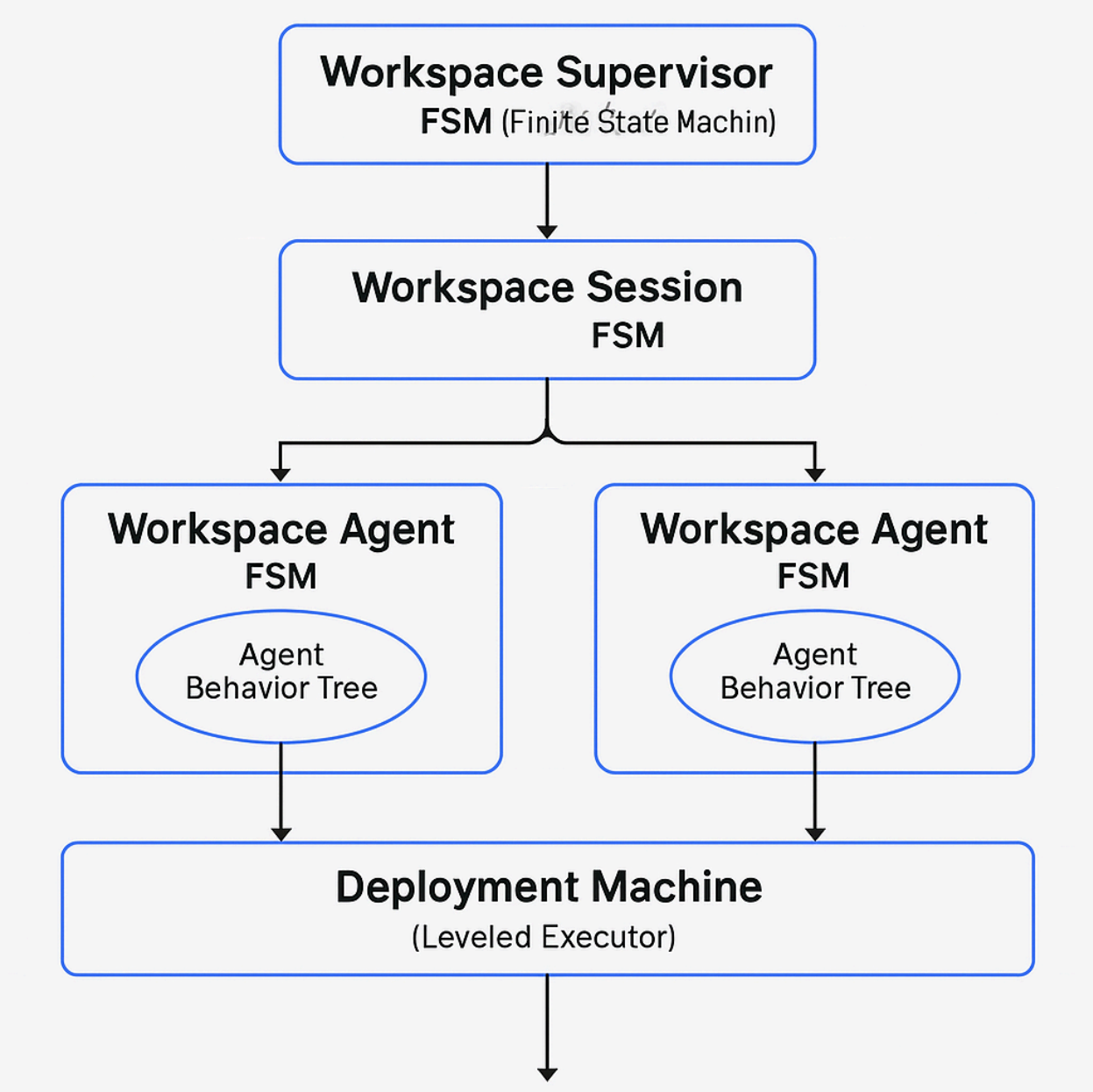
    d. `Taxonomy`
        i. `Tags`
        ii. `Classifications`
        iii. `Relationships`
22. The `WorkspaceSupervisor` has full visibility on all signals, agents, insights, workflows, people, and taxonomy for a workspace.
23. When spawning a new `WorkspaceSession` to process a `Signal`
24. `Supervisor` to `Agent` communication is possible
25. `Agent` to `Agent` communication is possible within a Workspace
26. Workspace built-in workflows:
    a. These are available by default
    b. Gates
        i. Human approval—notification and action required via Tempest clients and email
        ii. Programmatic approval—configure a webhook for Tempest to call for approval
        iii. Agentic approval
    c. Built-in signals
        i. New workspace creation from `WorkspaceTemplate`
        ii. Manual workflow invocation

Design Principles

The patterns and beliefs we've gained from our craft and experience.

Diagrams


Context Layering
Organization prompt
Agent prompt
Workspace prompt
Signal prompt
System Architecture Hierarchy
WorkspaceSupervisor
WorkspaceSession
WorkspaceAgent
WorkspaceAgent
WorkspaceAgent



Concepts

Agentic Behavior Trees (ABT)

https://medium.com/@Micheal-Lanham/orchestrating-ai-agents-with-behavior-trees-the-missing-piece-in-agent-architectures-729c14a6d83b

https://github.com/cxbxmxcx/Agentic-Behavior-Trees

Token

A **token** is a unit of text that a language model processes. Tokens and calls have become the de facto unit of cost/usage measurement for AI products.

It can be:

- A **word**
- A **sub-word**
- A **character**
- Or even **punctuation or whitespace**

It depends on the **tokenizer** — the system that breaks raw text into these units.

> You can think of a token as a kind of encoded “chunk” of text that the model understands.

Tokens in NLP vs LLMs:

Era	Token Definition	Purpose	Examples
Classical NLP	Word or punctuation (sometimes whitespace too)	Linguistic unit for analysis	`"Hello, world!"` → `["Hello", ",", "world", "!"]`
Modern LLMs	Subword unit, optimized for computation	Efficient input/output to model	`"unbelievable"` → `["un", "believ", "able"]`

https://platform.openai.com/tokenizer - OpenAI token previewer (type text, get token count)

Needs Clarification

🔍 Areas for Additional Definition

1. Agent Topology & Composition

The architecture of agent networks and hierarchies requires further definition in these areas:

- The specific mechanics of **agent-to-agent delegation and messaging**
- The `WorkspaceSupervisor`'s capabilities regarding agent composition trees and sub-agent management

2. Prompt Runtime Model

The prompt resolution process needs additional specification:

- The direction of prompt merging
- Runtime prompt string construction process
- Capabilities for prompt override and extension between agents

3. Signal Re-Invocation & Replay

- Deterministic replay capabilities for past sessions
- Required signal metadata for system analysis and reconstruction

4. Workflows vs Agents

- The relationship between deterministic `Workflow` and agent execution
- Workflow step integration within agent processes
- Signal context derivation from workflow outputs

5. Workspace Lifecycle

- Definition of workspace states (initialization, active state, hibernation)
- Workspace mutation capabilities via signals or agents

6. Human-in-the-Loop

- Integration points for human input and response handling during agent sessions, particularly regarding `People`, `Watchers`, and `Owners`

Ideas

Enterpise Model Context Protocol Extension

Agent/collaborator provenance

Rebrand/launch as navigator

Atlas Technical Design

Class & Object Design

```
// Atlas "Scopes" encapsulate the context, memory, and conversation
// for a AI-powered use case. These scopes can be hierachal in nature, which
// create a sort of call stack for context, memory, and messages.
interface IAtlasScope {
  id: string
  parentScopeId?: string
  supervisor?: IWorkspaceSupervisor

    context: ITempestContextManager
    memory: ITempestMemoryManager
    messages: ITempestMessageManager

    prompts: {
        system: string
        user: string
    }
```

```
    gates: []IAtlasGate

    newConversation(): ITempestMessageManager
    getConversation(): ITempestMessageManager
    archiveConversation()
    deleteConversation()
}

// Describes a generic interface for agent communication.
// The API will likely be similar to calling a model API (like Claude or GPT)
// but also descriptive information
//
// Our first-party agents will be instances of Atlas Scopes, while remote agents
// (self-hosted or third party) will be wrapped in a supervisor agent with an
// Atlas Scope
interface IAtlasAgent extends IAtlasScope {
    name()
    nickname()
    version()
    provider()
    purpose()
    prompts()
    scope()
    controls()
}

// The Atlas Decision Graph is a class for creating and managing
// agentic behavior trees:
// https://medium.com/@Micheal-Lanham/introduction-to-autonomous-assistant
interface IAtlasDecisionGraph {
}

// Gates are the policies for constraining the input, output, and
// abilility of agents. First party agents will be able to deeply respect
// gates beyond input/output validation, while remote-to-Tempest agents
```

```
// (self-hosted or third-party) will have to rely on input/output validation
// and rejection based on evaluation of that
interface IAtlasGate extends IAtlasDecisionGraph {
}

// A Tempest Workspace represents the virtual runtime for
// a network of humans and agents
//
// It extends Atlas Scopes
interface IWorkspace extends IAtlasScope {
  public members: IWorkspaceMember
  public messages: ITempestMessageManager
  public signals: Record<string, IWorkspaceSignal>
  public agents: Record<string, IWorkspaceAgent>
  public workflows: Record<string, IWorkspaceWorkflow>
  public sources: Record<string, IWorkspaceSource>
  public actions: Record<string, IWorkspaceAction>

  private supervisor: IWorkspaceSupervisor
  private activeSessions: []IWorkspaceSession
  private library: IWorkspaceLibrary

  private config: {
  }

  private acls: {
  }

  private artifacts: {
  }

  addSignal(IWorkspaceSignal): Error
  addAgent(IWorkspaceAgent): Error
  addWorkflow(IWorkspaceWorkflow): Error
  addSource(IWorkspaceSource): Error
  addAction(IWorkspaceAction): Error
```

```
  currentActiveSessions(): []IWorkspaceSession
  getAllArtifacts(): []IWorkspaceArtifacts

  snapshot()
}

// Workspace Artifacts are the atomic, portable, results of a workspace agent
// or workflow
// For example, an infrastructure resource like a VM,
// or even a PDF or excel spreadsheet.
interface IWorkspaceArtifact {}

// A Workspace Document is a higher level output resulting from
// a WorkspaceSession.
// Today in Tempest, our project's deployment page would be a document
// Or, a nicely formatted test report for Playwright could be a document
// or a RCA document
interface IWorkspaceDocument {}

// The workspace libray is an organized and explorable collection
// of artifacts and documents created by workspace agents and workflows
interface IWorkspaceLibrary {
    documents []IWorkspaceDocuments
    artifacts []IWorkspaceArtifact
}

// Workspace supervisors are critical to the secure and safety of
// the workspace sandbox.
//
// The WorkspaceSupervisor is a transparent agent—-it is automatically
// created by default for any workspace and is the only workspace with access
// to the full context, memory, and configuration of the workspace.
// It's job is to translate user prompts into structured plans (ABTs),
// process signals when they arrive, and launch sessions with the
// subset of context, scope, and prompting necessary for success.
```

```
// It also serves as the arbiter of communication across agents, as agents
// will not be able to talk directly to each other (decision not locked down)
interface IWorkspaceSupervisor extends IAtlasScope, IWorkspaceAgent {
}

// WorkspaceSessions are the fundamental unit of execution for a workspace.
// A session is a M:M execution of signals to agents, where the intended
// behavior is provided a priori from a human user.
interface IWorkspaceSession extends IAtlasScope {
  signals: {
    triggers: []IWorkspaceSignal
    callback: IWorkspaceSignalCallback
  }

  agents?: []IWorkspaceAgent
  workflows?: []IWorkspaceWorkflows
  sources?: []IWorkspaceSources

  private plan: IWorkspaceSessionPlan
  private artifacts: IWorkspaceArtifact

  start()
  cancel()
  progress()
  summarize()

  getArtifacts()
}

interface IWorkspaceSessionPlan extends IAtlasDecisionGraph {
}

interface IWorkspaceSignal extends IAtlasScope {
   public provider {
      id: string
      name: string
```

```
  }
}

interface IWorkspaceSignalCallback extends IAtlasDecisionGraph {
}

// An agent working within a workspace.
interface IWorkspaceAgent extends IAtlasAgent {
   status: string
   host: string
}

// Humans who have access to a workspace. This will be
// extended later with additional roles and capabilities.
interface IWorkspaceMember {
   role WorkspaceMemberRole
}

enum WorkspaceMemberRole {
   owner = 1
   watcher
}

// Context for a given scope, such as a workspace, agent, or signal.
interface ITempestContextManager {
  add(ITempestContext)
  remove(ITempestContext)
  search(string)
  size()
}

// An entry in context.
interface ITempestContext {
  source: {
    type: string
    id: string
```

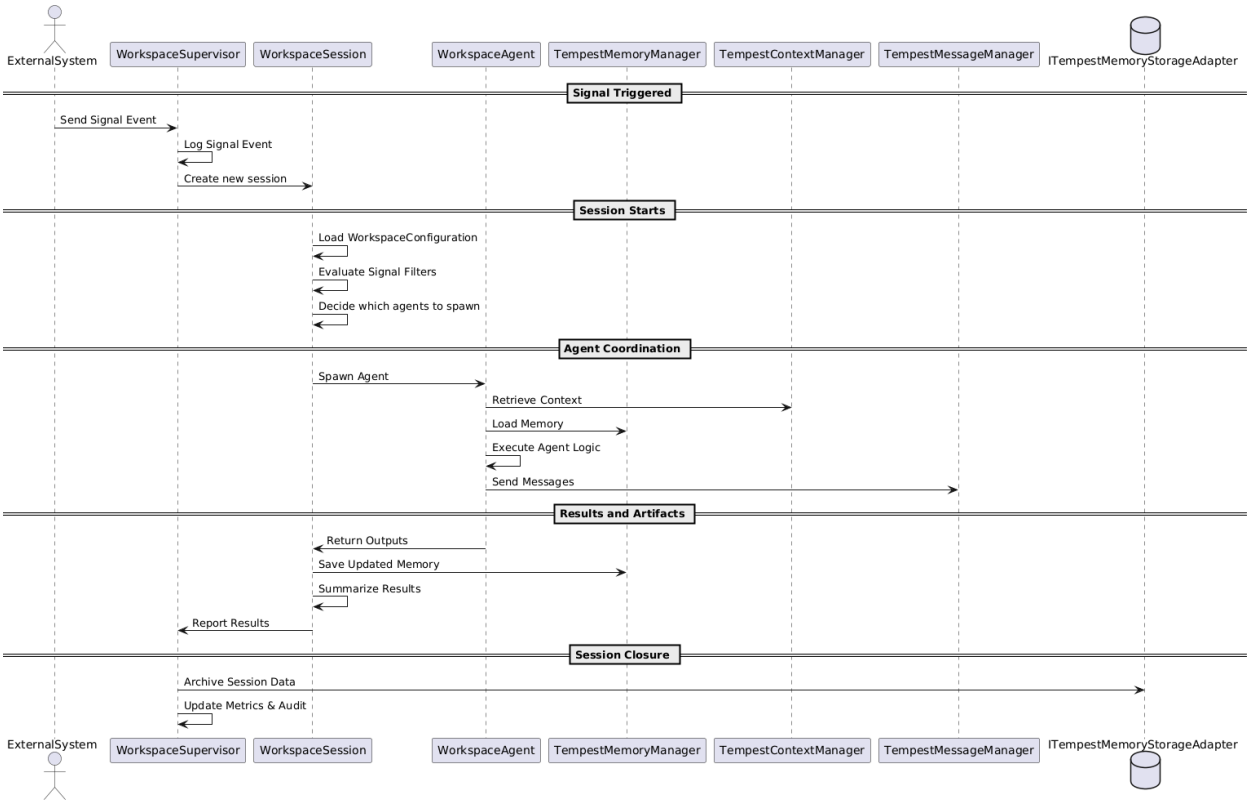
```
  }
  detail: string
}

interface ITempestMemoryManager {
  private store: ITempestMemoryStorageAdapter

  remember()
  recall()
  summarize()
  size()
  forget()
}

interface ITempestMessageManager {
   history []ITempestMessage

   newMessage(string)
   editMessage(id: string)
}

interface ITempestMemoryStorageAdapter {
    commit()
}

interface ITempestMessage {
   public promptUser: "human" | "agent" | "system"
   public id: string
   public message: string
}
```

Workspace Runtime Coordination Model

1. **WorkspaceSupervisor** is the root.
    - It can spawn **WorkspaceSessions** in response to signals.

- It holds global context/memory/messaging.

2. **WorkspaceSessions** are the main execution units.
   - Each session:
     - Has its own context/memory/messages (scoped)
     - Spawns agents and workflows
     - Coordinates triggers (signals) and callbacks
3. **Agents** are stateless at the core but can request context/memory from their parent scope.
   - They might share or isolate context depending on the session plan.
   - They produce messages and optionally artifacts.
4. **Signals** are event triggers.
   - Each has a provider.
   - Can trigger sessions.
   - May have callback graphs for post-processing or chaining.
5. **Workflows** are deterministic pipelines — presumably invoked by sessions or agents.



CLI interface

Tempest Navigator CLI

```
// root command
atlas

// list all sessions
atlas ps

// list
```